# Secretos en RAM: memoria volátil y no volátil

Proyecto 3 · Grupo 3 · ITI922 Seguridad de TI II · Universidad Técnica Nacional

**Raúl Quesada Morera**

**Justin Rodríguez González**

**Cristian Rojas Morales**

**Keilor Madrigal Ramírez**

# Volátil no significa que desaparezca al instante

## Jerarquía: lo más rápido es volátil

**Caché de CPU**
nanosegundos — Volátil

**RAM (DRAM)**
decenas de ns — Volátil

**Disco y memoria flash**
milisegundos — No volátil

**NVRAM del TPM**
persistente — No volátil

## Lo que hay que recordar

- Volátil: necesita energía para conservar el dato.
- La RAM no se borra de inmediato: la remanencia dura segundos o minutos.
- Si falta RAM, el sistema envía páginas al swap, en el disco, y ahí el secreto sobrevive al apagado.

# Cold boot: la RAM recuerda tras cortar la energía

1

### Equipo encendido

La llave de disco está en la RAM

2

### Corte de energía

Reinicio brusco, sin limpiar la RAM

3

### Remanencia

Las celdas pierden carga poco a poco

4

### Copia de la RAM

Arranque desde USB y búsqueda de llaves

**2,5 s – más de 1 min**

pérdida total a temperatura normal

**≥ 99,9 % de los bits**

se conservan a −50 °C tras 1 minuto

**Apagar por completo**

defensa más simple: no suspender ni hibernar

# El cifrado de disco necesita la llave en la RAM

**Frase de paso**

La escribe el usuario

**Llave maestra**

Guardada cifrada en el encabezado LUKS

**RAM del núcleo**

device-mapper usa la llave en claro

**Disco**

Datos cifrados en reposo; archivos legibles al usarlos

**Mientras el volumen está abierto, la llave tiene que estar en la RAM.**

Por eso un volcado de memoria o un cold boot la expone, aunque el disco esté cifrado.

**El TPM no protege una llave ya cargada en la RAM.**

Halderman et al. (2009) recuperaron llaves de equipos encendidos o recién apagados.

# Swap, hibernación y volcados: la RAM llega al disco

## 1 Swap

**Qué es**
Páginas expulsadas de la RAM al disco.

**Riesgo**
El secreto sobrevive al apagado.

**Medida**
mlock y swap cifrada.

## 2 Hibernación

**Qué es**
Imagen completa de la RAM en disco.

**Riesgo**
Incluye las llaves en uso.

**Medida**
Apagar por completo.

## 3 Volcado (core dump)

**Qué es**
Archivo con la memoria de un proceso, por una falla o con gcore.

**Riesgo**
Contraseña legible (CWE-14).

**Medida**
Desactivar volcados y limitar permisos.

# Buenas prácticas: borrar no es tan simple

## En C

- memset puede eliminarse (dead store): usar explicit_bzero.
- mlock evita la swap, pero no impide leer el secreto en uso.
- read(2) evita la copia en el búfer de stdio.

## En Python

- str es inmutable: del solo quita la referencia.
- bytearray se puede poner en ceros…
- …solo si el secreto nunca pasó por bytes o str.

**Resultados de nuestra práctica (copias de la contraseña en memoria)**

**2 → 0** C: vulnerable frente a seguro2 (read, mlock, explicit_bzero)

**3 → 3** Python str: del y gc.collect() no eliminan las copias

**1 → 0** Python bytearray leído con os.readv

# El TPM mezcla memoria volátil y no volátil

### Objeto transitorio

`0x80xxxxxx`

Memoria volátil

**Se pierde**

### Objeto persistente

`0x81xxxxxx`

Memoria no volátil (tpm2_evictcontrol)

**Sobrevive**

### Índice NV

`0x01xxxxxx`

Memoria no volátil, con políticas de acceso

**Sobrevive**

### Registro PCR

`PCR 0–23`

Memoria volátil

**Vuelve al valor inicial**

**En la práctica (TPM emulado):** tras reiniciar, sobrevivieron el persistente 0x81010001 y el índice NV 0x01500016; se perdieron el transitorio y el PCR 16.

# Caso real · Heartbleed: qué pasó

1

### Origen

Al actualizar OpenSSL para TLS 1.2 (2012) se agrega la extensión heartbeat

2

### Descubrimiento

CVE-2014-0160: se detecta mediante fuzzing

3

### Impacto

Cada solicitud leía hasta 64 kB del servidor sin dejar rastro

4

### Respuesta

Actualizar OpenSSL, reemitir certificados y cambiar contraseñas

## 64 kB

de memoria leídos por cada solicitud

## Sin rastro

el servidor no registraba la lectura

## Fuga de memoria

se leyó la RAM; el cifrado no se rompió

# Caso real · Cómo funcionaba el fallo

**Cliente**

Envía un latido con 4 bytes de carga, pero declara 64 kB

**Servidor OpenSSL**

Confía en la longitud declarada y no la verifica

**Respuesta**

Devuelve hasta 64 kB de memoria contigua del servidor

## Por qué ocurrió

- OpenSSL confiaba en la longitud declarada por el cliente.
- Si la solicitud mentía, devolvía memoria contigua.

## Qué se expuso

- Llaves privadas del servidor.
- Credenciales y otros datos.

# Caso real · Qué se aprende

| | | |
|---|---|---|
| 1 | **Un defecto de lectura basta** | Quien lee la RAM obtiene las llaves sin romper el cifrado. |
| 2 | **Borrar la variable no alcanzaba** | Un servidor necesita sus llaves en cada conexión. |
| 3 | **Reducir el daño** | Rotar llaves, aislarlas en otro proceso o módulo de hardware y parchear rápido. |
| 4 | **No confiar a ciegas** | Conocer y probar el código de terceros. |